Ho Chi Minh City University of Technology
Faculty of Electrical – Electronics Engineering
Integrated Circuit Design Major



EE3215 - SYSTEM ON CHIP (SoC)

Project Milestone 1

HW/SW Partitioning of a 3×3 2-D Convolution Application

Instructor: Hieu Nguyen

Email: hieunt@hcmut.edu.vn

October 25, 2026

Contents

1 Introduction **2**

1.1 3×3 2-D Convolution . . . 2

1.2 HW/SW Co-design Question . . . 2

1.3 Project Learning Objectives . . . 3

2 Requirements and implementation procedure **4**

2.1 Requirements . . . 4

2.2 Single-Cycle RISC-V CPU and Memory-Mapped I/O . . . 4

2.2.1 Required Global Register Map . . . 5

2.2.2 Required FPGA I/O Demonstration . . . 6

2.3 Required HW/SW Architectures . . . 6

2.3.1 Implementation B: RISC-V CPU + One Multiplier . . . 6

2.3.2 Implementation C: RISC-V CPU + 3×3 Convolution Accelerator . . . 7

2.4 Implementation Procedure . . . 7

2.4.1 Implementation A: Software-Only Assembly Baseline . . . 7

2.4.2 Implementation B: RISC-V CPU + One Multiplier . . . 8

2.4.3 Implementation C: RISC-V CPU + 3×3 Convolution Accelerator . . . 8

2.4.4 Single-Cycle RISC-V MMIO Integration . . . 8

2.4.5 Simulation and Verification . . . 9

2.4.6 Synthesis and Architecture Measurements . . . 9

2.4.7 Architecture Comparison . . . 9

2.4.8 Reports and Design Defense . . . 10

3 Grading Criteria **11**

1 Introduction

1.1 3×3 2-D Convolution

Two-dimensional convolution is widely used in image processing and machine-learning workloads. A small convolution kernel is moved across an input image. At every valid output position, the corresponding image samples are multiplied by the kernel coefficients and accumulated to produce one output value.

For a 3×3 kernel K and an input image I, this project uses **valid convolution**, **stride 1**, and **no padding**. The output is

$$Y[r, c] = \sum_{k_r=0}^{2} \sum_{k_c=0}^{2} I[r + k_r, c + k_c]\, K[k_r, k_c]. \tag{1}$$

For an input image of size $H \times W$, the output size is

$$H_{out} = H - 2, \qquad W_{out} = W - 2. \tag{2}$$

Therefore, the number of output pixels is

$$N_{out} = (H - 2)(W - 2), \tag{3}$$

and every output pixel requires nine multiplications. The total number of multiplications is

$$N_{mul} = 9(H - 2)(W - 2). \tag{4}$$

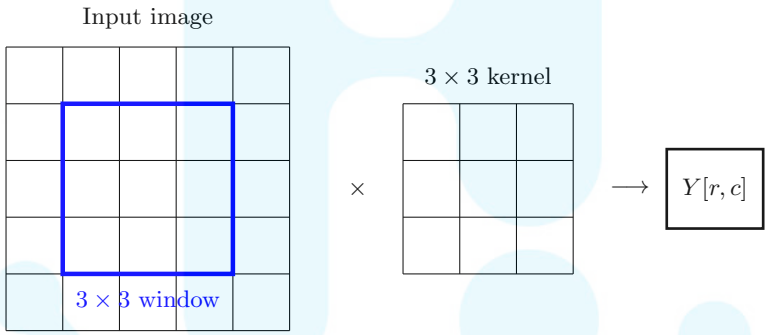


Figure 1: A 3×3 convolution window produces one output pixel

1.2 HW/SW Co-design Question

The purpose of this milestone is not only to implement convolution. Students must compare three implementations of the **same application**, using the **same input image**, **same kernel**, and **same arithmetic precision**. The processor used throughout the project is a **single-cycle RISC-V CPU**, and all processor software submitted for the project must be written in **RISC-V assembly language**.

The central question is:

How does moving computation and control from RISC-V assembly software into dedicated hardware change execution time, CPU involvement, communication overhead, and hardware cost?

Three implementations are required.

Implementation A – Software-only baseline. The single-cycle RISC-V CPU executes the complete 3×3 convolution in assembly. The CPU performs loop control, address generation, multiplication using the processor-supported software/instruction sequence, accumulation, and output storage. This implementation provides the cycle-count and functional baseline.

Implementation B – RISC-V CPU + One Multiplier. The RISC-V CPU still executes all convolution loops in assembly, but each multiplication is offloaded to one memory-mapped hardware multiplier. The CPU writes multiplier operands and control registers using store instructions, polls status using load instructions, reads the product, performs accumulation in software, and stores the output pixel.

Implementation C – RISC-V CPU + 3×3 Convolution Accelerator. The RISC-V CPU uses assembly to load the image and kernel into the accelerator, configure image dimensions, write `START`, poll `DONE`, and read the output image. The dedicated convolution IP autonomously performs loop control, address generation, multiplication, accumulation, and output generation.

To make Implementations B and C comparable, both mandatory hardware baselines use exactly **one custom hardware multiplier**. Their performance difference should therefore expose the impact of HW/SW partitioning, control placement, data movement, and hardware specialization rather than simply a larger multiplier count.

The CPU, GPIO, LEDs, multiplier, and convolution accelerator are connected through a custom **memory-mapped I/O (MMIO) address decoder**. Standard on-chip buses such as TileLink, AXI, and APB are intentionally postponed to later milestones.

1.3 Project Learning Objectives

After completing this milestone, students should be able to:

- analyze the dataflow and loop-carried dependencies of a 2-D convolution kernel
- identify alternative HW/SW partitioning boundaries
- implement convolution control software in RISC-V assembly on a single-cycle processor
- map the convolution algorithm into datapath, registers, memories, and an FSMD controller
- design a custom MMIO register map and address decoder for GPIO, LEDs, and custom hardware IP
- verify accelerator results against the software-only assembly baseline
- compare latency, CPU involvement, hardware resources, and communication overhead among three implementations

2 Requirements and implementation procedure

2.1 Requirements

Design and verify three implementations of a 3×3 2-D convolution application using a **single-cycle RISC-V CPU**: (1) a software-only baseline, (2) a CPU-controlled one-multiplier peripheral, and (3) a dedicated convolution accelerator. All processor software must be written in **RISC-V assembly language**. CPU-to-I/O and CPU-to-IP communication uses a custom memory-mapped I/O address decoder; TileLink, AXI, APB, or another standard on-chip bus is **not** used in Milestone 1.

The following functional assumptions are used throughout the project:

- convolution type: valid 3×3 convolution
- stride: 1
- padding: none
- input pixels: unsigned 8-bit values
- kernel coefficients: signed 8-bit values
- accumulator and output: signed 32-bit values
- each group must correctly demonstrate at least 8×8 and 16×16 input images
- both Implementation B and Implementation C must use the same test images and kernels
- the mandatory baseline for each problem uses exactly one hardware multiplier

A group may implement a parameterized image size, additional multipliers, line buffers, or a sliding-window architecture as an extension. However, the mandatory comparison must include the one-multiplier baseline for both implementations.

2.2 Single-Cycle RISC-V CPU and Memory-Mapped I/O

All software executed by the processor in this project must be written in **RISC-V assembly language**. The processor used in Milestone 1 is a **single-cycle RISC-V CPU**. Each supported instruction completes in one processor clock cycle according to the CPU implementation supplied or approved for the course.

The processor must support, at minimum, the integer instructions required for control flow, address calculation, arithmetic, and memory-mapped I/O. Typical required instructions include `lw`, `sw`, `lb`/`lbu`, `sb`, integer arithmetic/logical instructions, comparison/branch instructions, and jump instructions. The exact supported ISA subset must be documented by each group.

The CPU communicates with the external I/O and both hardware accelerators using **memory-mapped I/O (MMIO)** implemented by a simple address decoder. TileLink, AXI, APB, or another standard on-chip bus is **not** used in Milestone 1.

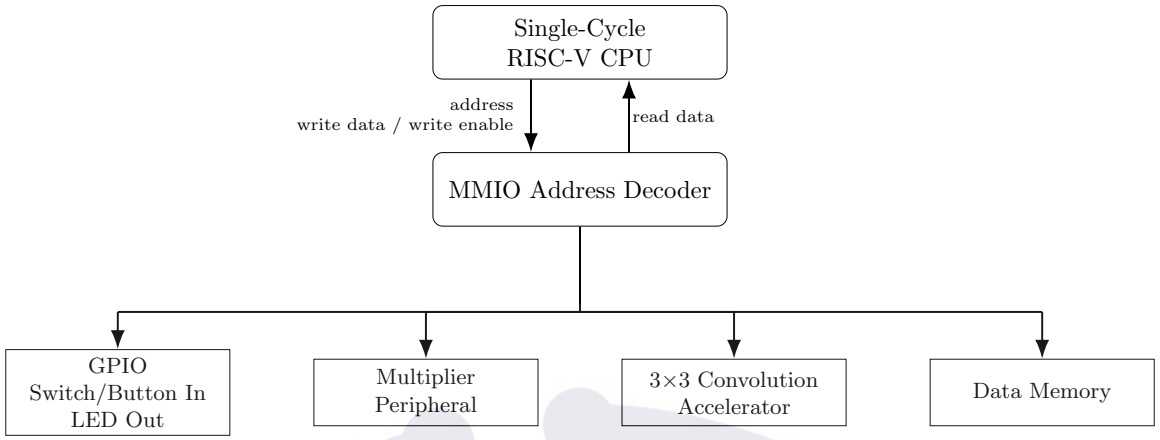


Figure 2: Milestone 1 single-cycle RISC-V system with a custom MMIO decoder

2.2.1 Required Global Register Map

Students must implement and document a register map equivalent to Table 1. The addresses below are the required default addresses unless the instructor approves an alternative map. All registers are accessed by RISC-V load/store instructions.

Address	Register	Access	Width	Function
0x4000_0000	GPIO_IN	RO	32	Read switches/buttons or other FPGA input signals.
0x4000_0004	LED_OUT	RW	32	CPU-visible output register connected to board LEDs. Only implemented LED bits need physical pins.
0x4000_0100	MUL_A	RW	32	Multiplier operand A. The valid operand width and sign-extension rule must be documented.
0x4000_0104	MUL_B	RW	32	Multiplier operand B.
0x4000_0108	MUL_CTRL	RW	32	Bit 0: START. Writing 1 launches one multiplication. Other bits reserved.
0x4000_010C	MUL_STATUS	RO	32	Bit 0: BUSY; Bit 1: DONE. Other bits reserved.
0x4000_0110	MUL_RESULT	RO	32	Multiplication result.
0x4000_0200	CONV_CTRL	RW	32	Bit 0: START. Writing 1 starts one complete convolution task.
0x4000_0204	CONV_STATUS	RO	32	Bit 0: BUSY; Bit 1: DONE.
0x4000_0208	IMG_WIDTH	RW	32	Input image width.
0x4000_020C	IMG_HEIGHT	RW	32	Input image height.
0x4000_0210	IMG_ADDR	RW	32	Address/index used to access accelerator input-image storage.
0x4000_0214	IMG_DATA	RW	32	Image data port. Low implemented bits contain the pixel value.
0x4000_0218	KER_ADDR	RW	32	Kernel coefficient index 0–8.
0x4000_021C	KER_DATA	RW	32	Kernel coefficient data. Low implemented bits contain the signed coefficient.
0x4000_0220	OUT_ADDR	RW	32	Address/index used to read accelerator output storage.
0x4000_0224	OUT_DATA	RO	32	Signed convolution output corresponding to OUT_ADDR.

Table 1: Required Milestone 1 MMIO register map.

2.2.2 Required FPGA I/O Demonstration

Before integrating either accelerator, students must demonstrate correct MMIO operation using board I/O:

Switch/Button → CPU load → CPU store → LED

The assembly program must repeatedly read `GPIO_IN` with a load instruction and write a derived value to `LED_OUT` with a store instruction. This demonstration verifies the single-cycle CPU data path, load/store behavior, MMIO address decoding, and FPGA I/O mapping.

A minimal assembly sequence may follow the structure below. Register allocation may be changed.

```
    li   t0, 0x40000000      # GPIO_IN
    li   t1, 0x40000004      # LED_OUT
loop:
    lw   t2, 0(t0)           # read switches/buttons
    sw   t2, 0(t1)           # drive LEDs
    jal  x0, loop
```

2.3 Required HW/SW Architectures

2.3.1 Implementation B: RISC-V CPU + One Multiplier

In Implementation B, the hardware accelerator is intentionally fine-grained. The only arithmetic operation offloaded to custom hardware is multiplication. The **single-cycle RISC-V CPU** retains responsibility for the complete convolution algorithm, and the control program is written entirely in RISC-V assembly. The CPU accesses the multiplier through the MMIO registers listed in Table 1.

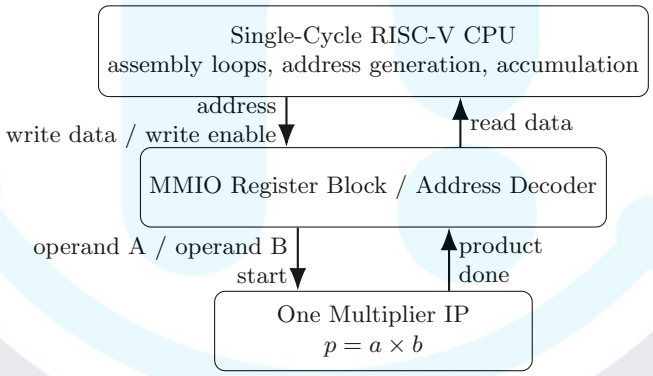


Figure 3: Implementation B: RISC-V CPU + One Multiplier

At the software-visible level, the multiplier is controlled through `MUL_A`, `MUL_B`, `MUL_CTRL`, `MUL_STATUS`, and `MUL_RESULT` in Table 1. Internally, the register block may drive the multiplier signals shown in Table 2.

Name	Direction	Width	Description
`mul_a`	CPU → IP	16	Operand A; zero/sign extension must be documented
`mul_b`	CPU → IP	16	Operand B
`mul_start`	CPU → IP	1	Starts one multiplication
`mul_done`	IP → CPU	1	Indicates valid multiplication result
`mul_result`	IP → CPU	32	Signed multiplication result

Table 2: Internal multiplier-register-block interface for Implementation B.

For each output pixel, the CPU performs nine multiplier transactions and accumulates the nine products in software. Thus, the hardware multiplier has little knowledge of the application structure.

2.3.2 Implementation C: RISC-V CPU + 3×3 Convolution Accelerator

In Implementation C, the HW/SW partition boundary is moved upward. The single-cycle RISC-V CPU executes an assembly control program that configures the accelerator through MMIO and offloads an entire convolution task. The baseline accelerator still contains only one multiplier, but nested-loop control, address generation, and accumulation are moved into hardware.

For the assignment handout, a **top-level architecture view is sufficient**. The handout does not need to show the detailed internal organization of the 3×3 convolution accelerator, and a baseline FSMD figure is not required here. Each student group may define and document its own internal microarchitecture in the report and design defense.

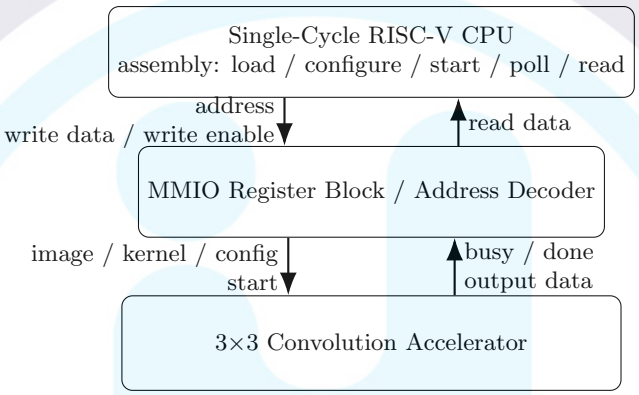


Figure 4: Implementation C: RISC-V CPU + 3×3 Convolution Accelerator

At the software-visible level, the CPU communicates with the convolution accelerator through the MMIO register set in Table 1. The CPU is responsible for loading image data and kernel coefficients, configuring problem dimensions, asserting `start`, polling `busy`/`done`, and reading the output data.

Compared with Implementation B, Implementation C changes the granularity of HW/SW interaction: the CPU no longer invokes hardware for every multiply operation. Instead, the CPU launches a larger computation, while the convolution IP performs the convolution task internally and returns the result set to software.

2.4 Implementation Procedure

2.4.1 Implementation A: Software-Only Assembly Baseline

Before using any accelerator, students must implement the complete convolution algorithm in **RISC-V assembly** and execute it on the single-cycle CPU. This implementation is the software baseline for correctness and cycle-count comparison.

The assembly program must:

- execute all nested convolution loops on the CPU
- perform image/kernel address calculation in assembly
- perform multiplication using the processor-supported software/instruction sequence approved for the course
- accumulate each output pixel in CPU registers
- store the full $(H-2) \times (W-2)$ result to data memory

- signal PASS/FAIL or execution status using the memory-mapped LED output

No C program may be used as the submitted processor software. An external Python/C model may be used only as a testbench-side golden model for checking expected numerical results.

2.4.2 Implementation B: RISC-V CPU + One Multiplier

Students shall implement the fine-grained architecture described in Section 2.3.1. The CPU must execute the convolution loops and invoke the multiplier for each image/kernel pair.

The CPU software must perform the following operations for each output pixel:

1. initialize the software accumulator
2. determine the required image and kernel indices
3. send one pair of operands to the multiplier
4. assert the multiplier start signal
5. wait until the multiplier reports completion
6. read the multiplication result
7. add the multiplication result to the software accumulator
8. store the completed output pixel

For a valid output image, the CPU therefore initiates exactly N_{mul} multiplication operations as defined in Equation 4.

2.4.3 Implementation C: RISC-V CPU + 3×3 Convolution Accelerator

Students shall implement the coarse-grained architecture described in Section 2.3.2. The accelerator must contain, at minimum:

- local storage for the input image or an equivalent directly accessible image storage structure
- storage for the nine kernel coefficients
- one multiplier
- a 32-bit accumulator register
- row/column and kernel-tap counters
- address-generation logic
- output storage or an output read interface
- an FSMD controller

The CPU shall load the image and kernel into the accelerator, configure image dimensions, assert `start`, wait for completion, and read the output. During convolution execution, the accelerator must autonomously perform the nested-loop sequencing and accumulation.

2.4.4 Single-Cycle RISC-V MMIO Integration

The CPU used in all three implementations is a **single-cycle RISC-V processor**. All CPU software must be written in RISC-V assembly. The CPU accesses GPIO, LEDs, the multiplier peripheral, and the convolution accelerator only through load/store instructions to the MMIO addresses defined in Table 1.

Students must implement the address decoder and show how CPU address, write data, write enable, and read data are routed to the selected peripheral register. Standard buses such as TileLink, AXI, and APB are intentionally excluded from this milestone.

The following integration sequence is mandatory:

1. verify CPU instruction execution using a simple assembly program
2. verify data-memory load/store operations

3. verify `GPIO_IN` → CPU → `LED_OUT`
4. verify the mapped multiplier registers
5. verify the mapped convolution-accelerator registers
6. run the complete assembly applications for all three implementations

2.4.5 Simulation and Verification

To verify both architectures, students must:

- compare every output pixel from Implementations B and C against the software-only RISC-V assembly baseline
- test at least one 8×8 image and one 16×16 image
- test at least three different 3×3 kernels
- include at least one kernel with negative coefficients
- include at least one randomly generated image
- show a waveform demonstrating single-cycle CPU load/store access to GPIO/LED and correct MMIO control for both Implementation B and Implementation C

Recommended kernels include blur, sharpen, and edge-detection filters. Students may use other kernels provided their coefficient ranges are compatible with the specified signed 8-bit format.

The waveform for Implementation B should show, at minimum, multiplier operands, `start`, `done`, and multiplication result. The waveform for Implementation C should show, at minimum, `start`, `busy`, `done`, the current FSMD state, output-pixel indices, kernel-tap index, and accumulator value.

2.4.6 Synthesis and Architecture Measurements

Students must synthesize the two hardware designs using the synthesis flow specified by the instructor. At minimum, report:

- total execution cycles
- number of CPU–hardware interactions
- number of multipliers
- register count or equivalent sequential-resource report
- combinational area or equivalent resource estimate
- maximum clock frequency or critical-path timing if available from the synthesis flow

To avoid misleading conclusions, students shall report two latency measurements:

1. **Compute latency:** time from the beginning of convolution computation until the last output is produced, excluding any common testbench initialization
2. **End-to-end latency:** includes CPU-to-IP data/configuration transfer required by the corresponding architecture, computation, and output retrieval

2.4.7 Architecture Comparison

The final report must explain the architectural reasons for the measured results. A table similar to Table 3 must be included.

Metric	A: Assembly Only	B: RISC-V CPU + One Multiplier	C: RISC-V CPU + 3×3 Conv. Accelerator
Processor	Single-cycle RISC-V	Single-cycle RISC-V	Single-cycle RISC-V
Processor software	Assembly	Assembly	Assembly
Custom HW multipliers	0	1	1
Who controls convolution loops?	CPU	CPU	Hardware FSMD
Who generates image/kernel addresses?	CPU	CPU	Hardware
Who performs accumulation?	CPU	CPU	Hardware accumulator
CPU MMIO accesses	GPIO/LED only	Measured	Measured
Total CPU cycles	Measured	Measured	Measured
End-to-end cycles	Measured	Measured	Measured
Hardware area/resources	Measured	Measured	Measured
CPU availability during convolution	None	Low	High after start
Correct result	PASS/FAIL	PASS/FAIL	PASS/FAIL

Table 3: Required architecture comparison.

Students must explicitly answer the following question in the report:

Why do the three implementations have different execution time and CPU utilization even when Implementations B and C both use exactly one custom hardware multiplier?

2.4.8 Reports and Design Defense

All groups must submit a report and conduct a design defense. The report must clearly present:

- application algorithm and software-only RISC-V assembly baseline
- dataflow/dependency analysis
- HW/SW partitioning of all three required implementations
- top-level architecture diagrams
- module hierarchy
- datapath and FSMD of the convolution accelerator
- the complete MMIO register map and address-decoder behavior
- RISC-V assembly listings, verification methodology, and representative waveforms
- synthesis/resource results
- quantitative comparison and design conclusions

During the defense, all members of the group may be asked to explain RTL, software, FSMD behavior, interface timing, and measurement results.

3 Grading Criteria

Your project will be evaluated based on the following criteria:

Criteria	**Grade**
Single-cycle RISC-V CPU operation, data-memory access, MMIO address decoder, and GPIO/LED demonstration	1.5 pts
Implementation A: correct software-only 3×3 convolution in RISC-V assembly	1.0 pt
Implementation B – RISC-V CPU + One Multiplier: correct assembly-controlled MMIO implementation	1.5 pts
Implementation C – RISC-V CPU + 3×3 Convolution Accelerator: correct datapath, local storage, address generation, and one-multiplier architecture	2.0 pts
Implementation C – RISC-V CPU + 3×3 Convolution Accelerator: correct FSMD/control design and MMIO programmer-visible registers	1.5 pts
Verification, cycle/resource measurements, and quantitative comparison of all three implementations	1.5 pts
Report quality, assembly-code explanation, and design defense	1.0 pt
Total	**10.0 pts**

Table 4: Grading Criteria